Accelerated array computing and code transformations for R

Sebastian Fischer[1] · Daniel Falbel · Martin Binder[1] · Bernd Bischl[1]

[1]LMU Munich, Munich Center for Machine Learning (MCML)

Motivation

anvl brings **compiled, GPU-accelerated array computing** to the R community: you write array programs in plain R, which anvl compiles to optimized machine code via the **OpenXLA** compiler stack — the same infrastructure behind JAX, TensorFlow, and PyTorch/XLA. anvl aspires to be for R what JAX is for Python.

① Execution: eager & JIT

Running example — a **linear-model forward pass**:

```
predict <- function(X, beta) X %*% beta
```

Eager mode (default): each operation is dispatched and run immediately, optimized in isolation — ideal for interactive use and debugging.

JIT mode: `jit()` traces the whole function into a computational graph, which the XLA compiler optimizes as a single unit — fusing operations, eliminating redundant buffers, and specializing to the actual shapes and dtypes — before emitting one cached executable.

```
y_hat <- predict(X, beta)  # eager: runs now
fast  <- jit(predict)      # trace + compile
y_hat <- fast(X, beta)     # cached executable
```

② Automatic differentiation

`gradient()` returns a new function that computes **reverse-mode gradients** of a plain R function — no derivations by hand, and it composes with `jit()`. Here we fit the linear model above by minimizing its **mean squared error**:

```
loss <- function(X, y, beta) mean((X %*% beta - y)^2)

grad <- gradient(loss, wrt = "beta")
g    <- grad(X, y, beta)   # gradient w.r.t. beta
```

What anvl can do

≈ 140 array operations. Construction, arithmetic, linear algebra (`nv_matmul`, `nv_qr`, `nv_svd`, `nv_chol`, `nv_solve`), reductions and statistics, random-number generation, and JIT-able control flow (`nv_while`).

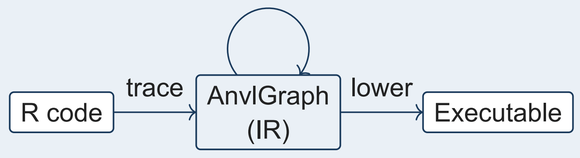
Build, in plain R. Neural networks, matrix factorizations, custom optimizers, gradient-based estimators, and Monte Carlo samplers — all expressed directly in R and run on CPU or GPU.

Why anvl? GPU acceleration without leaving R · a more intuitive and consistent array-computing API than base R · simplified CUDA install (only an NVIDIA driver needed) · JIT compilation sets it apart from **{torch}** · written primarily in R, so easy to extend and contribute to.

⚠️ **Scope:** anvl currently targets **dense** arrays — sparse matrices are not well supported yet. Best fit: dense data, regular shapes, and repeated or batched computation.

How it works

anvl compiles your code through **three code transformations**. Below we follow them on a tiny example function — `f <- function(x, y) x * y + 1` — calling each transformation directly and showing its output.



① Trace — R → AnvlGraph

```
trace_fn(f, args = list(
  x = nv_aval("f32", integer()),
  y = nv_aval("f32", integer())))
```

```
<AnvlGraph>
  Inputs:
    %x1: f32[]
    %x2: f32[]
  Body:
    %1: f32[] = mul(%x1, %x2)
    %2: f32[] = add(%1, 1:f32?)
  Outputs:
    %2: f32[]
```

② Transform — graph → graph

```
transform_gradient(
  graph, wrt = c("x", "y"))
```

```
<AnvlGraph>
  Inputs:
    %x1: f32[]   %x2: f32[]
  Constants:
    %c1: f32[]
  Body:
    %1 = mul(%x1, %x2)
    %2 = add(%1, 1:f32?)
    %3 = mul(%c1, %x2)
    %4 = mul(%c1, %x1)
  Outputs:
    %3: f32[]   %4: f32[]
```

③ Lower — graph → executable

```
stablehlo(graph)[[1]]
```

```
func.func @main
   (%0: tensor<f32>,
    %1: tensor<f32>) -> tensor<f32> {
 %2 = stablehlo.multiply %0, %1
        : tensor<f32>
 %3 = stablehlo.constant
        dense<1.0e0> : tensor<f32>
 %4 = stablehlo.add %2, %3
        : tensor<f32>
 return %4 : tensor<f32>
}
```

★ Application: randomized SVD for bioinformatics

Randomized SVD approximates the top-*k* singular triplets of a large matrix by sketching it onto a low-dimensional subspace with a random Gaussian matrix Ω, then taking an exact SVD of the small projection. It is a matmul-heavy, dense workload — XLA's sweet spot — so `jit()`-ing it on a GPU scales these decompositions to larger data.

```
rsvd <- jit(function(A, Omega) {
  Q <- nv_qr(nv_matmul(A, Omega))$Q   # sketch + orthonormalize
  B <- nv_matmul(nv_transpose(Q), A)  # project (small)
  s <- nv_svd(B)                      # exact SVD of B
  list(d = s$d, u = nv_matmul(Q, s$u), vt = s$vt)
})
```

⚠️ **For Bioconductor users:** sparse single-cell / genomics matrices would need to be densified first.

Benchmark figure

Placeholder — runtime of randomized SVD
(base R vs. anvl CPU vs. anvl GPU)
to be added once data is available.

Getting started

Install from r-universe — CPU works out of the box; NVIDIA GPU support is added via the `cuda12.8` package.

Runs on: CPU — Linux, Windows, Apple Silicon · NVIDIA GPU — Linux x86-64.

Status: early-stage, not yet on CRAN — feedback and contributions welcome.

github.com/r-xla/anvl **r-xla.github.io/anvl**

Supported by **MaRDI** — the Mathematical Research Data Initiative, an NFDI project funded by DFG grant 460135501.